

Score your AI instruction file

For CLAUDE.md, AGENTS.md, .cursorrules, or any AI assistant configuration file. Ten minutes, six categories, 100 points. Rubric on page 2.

File	Project	Date scored	Line count
CATEGORY	WHAT IT MEASURES	SCORE	
Structure & organization	Can the AI find information quickly? Clear hierarchy, quick start near the top, nothing critical buried.		/ 20
Accuracy & currentness	Commands and paths work as written; examples run; no stale dates or version claims; describes the code as it is.		/ 20
Comprehensiveness	Architecture, security, and troubleshooting covered; no half-written stubs.		/ 20
Actionability	Copy-paste-ready commands, do/don't pairs, concrete examples instead of abstract advice.		/ 20
Information balance	Detail proportional to how often it matters; long patterns split into referenced files; no design docs in disguise.		/ 10
Usability for AI	A quick start, a red-flags section telling the AI when to stop and ask, links out to deeper resources.		/ 10
Total			/ 100

90–100

In good shape. Re-score after major architectural changes, not on a calendar.

75–89

Usable. Treat your deductions as a ranked to-do list — accuracy problems first.

Below 75

Consider rewriting from a 50-line minimal file and growing it only where real friction appears.

Quick checks

- ☐ Has a "Last Updated" date? Remove it — dates rot and then lie.
- ☐ Pixel-level UI specs? Move to a design doc.
- ☐ Code style rules? Delegate to a linter; keep one line: run it.
- ☐ Security section missing? Add API keys, PII, privacy rules.
- ☐ No red-flags section? Write it first — it prevents confident wrong guesses.

Instruction math

Instructions in your file ≈ _____

Tool's own system prompt ≈ ~50

Total ≈ _____

Over ~200 total is past what current frontier models follow reliably. Split detailed content into separate files and reference them.

Line count bands:

Under 100 — probably too minimal · 100–500 — healthy range · 500–800 — ask hard questions · 800+ — almost certainly working against you

What earns points, what costs them

Score each category against these criteria, then write your deductions in the margin — they become your to-do list.

Structure & organization

20 points

EARNs POINTS

- Clear section hierarchy; a scan finds anything in seconds
- Quick start and critical rules near the top
- Table of contents once the file passes a few hundred lines

COSTS POINTS

- Critical rules buried mid-file
- The same guidance repeated in two sections

Accuracy & currentness

20 points

EARNs POINTS

- Every command and path works as written
- Examples describe formats ("ISO 8601 timestamp"), not literal moments
- Review is trigger-based (architecture changed), not calendar-based

COSTS POINTS

- A wrong or stale "Last Updated" date (this cost me 5 points)
- "Current state" claims nobody has verified
- Version numbers that outdate on the next release

Comprehensiveness

20 points

EARNs POINTS

- Architecture sketch an AI can orient by
- Security guidance: API keys, PII, privacy handling
- Troubleshooting for the errors that actually recur

COSTS POINTS

- Half-written stub sections ("TODO: session state")
- No security section at all
- Thin coverage of known failure modes

Actionability

20 points

EARNs POINTS

- Copy-paste-ready commands and snippets
- Do/don't pairs showing the pattern and the anti-pattern
- A concrete example wherever a rule could be misread

COSTS POINTS

- Abstract advice ("write good tests") with no example
- Rules that require judgment the AI doesn't have context for

Information balance

10 points

EARNs POINTS

- Detail proportional to how often the AI needs it
- Long patterns split into a referenced file, not inlined

COSTS POINTS

- Design documentation disguised as AI instructions — pixel specs, font sizes, color codes
- Style rules a linter should own
- Testing minutiae inlined at full length

Usability for AI

10 points

EARNs POINTS

- Quick start: install, run, test in a few lines
- Red-flags section: stop and ask when requirements are vague, multiple valid approaches exist, or a change touches schema or public APIs
- Pointers to deeper docs instead of duplicating them

COSTS POINTS

- No guidance on when the AI should ask instead of guess

How to use your score. The grade matters less than the deductions. Rank them — accuracy first, then comprehensiveness, then balance — and fix in that order. Re-score after the next major architectural change. The framework is tool-agnostic: the same 100 points apply whether the file is CLAUDE.md, AGENTS.md, or .cursorsrules.